Floating Point

How machines see numbers

Prof. Nipun Batra

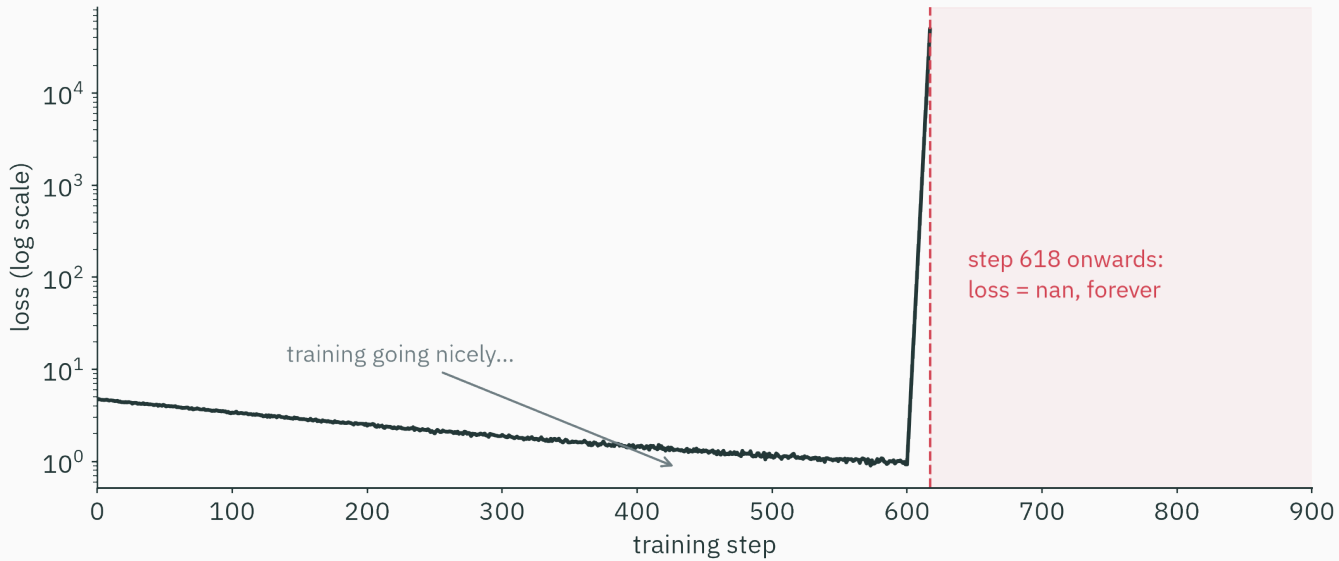
Mathematical Foundations for AI · IIT Gandhinagar

A softmax computation that returns nan

The numerical failure from Lecture 1

V VISUAL

617 steps of beautiful training, then nan forever:



Today we solve it — and learn the fix used inside **every** ML framework you will ever touch.

Exhibit B · a one-line scandal

First, type this into any Python prompt:

```
>>> 0.1 + 0.2 == 0.3
False

>>> 0.1 + 0.2
0.30000000000000004
```

Not a Python bug. The same happens in C, Java, Rust, JavaScript, Excel, your calculator app…

Guess before we continue: is `0.5 + 0.25 == 0.75` also `False`? Commit to an answer — we'll resolve it (and you'll see *exactly why*) within the hour.

Learning outcomes

By the end of this lecture you will be able to:

1. Explain why fixed-size **integers and fixed point** can't serve ML — and what dynamic range is.
2. Decode and encode **IEEE-754 float32** by hand (sign / exponent / mantissa).
3. Reason about the **unevenly spaced** float number line: machine epsilon, `x + 1 == x`.
4. Predict **overflow / underflow**: why `exp(89)` breaks float32.
5. Spot **catastrophic cancellation** and rewrite formulas to dodge it.
6. Derive and apply the **log-sum-exp trick** and the numerically stable softmax.
7. Choose between **float32 / float16 / bfloat16** — and say why DL picked bfloat16.

Before floats: integers and fixed point

Integers · exact, but boxed in

A 32-bit integer stores any whole number in $[-2^{31}, 2^{31}-1] \approx \pm 2.1 \times 10^9$ — **exactly**.

```
>>> 2**31 - 1          # largest int32
2147483647
```

- Arithmetic on integers is *perfect*: no rounding, ever.
- But there are no fractions — and beyond the box, classic C/NumPy `int32` **wraps around** to negative numbers.

Q. So how do we get decimals? First idea: just *fix* where the point goes.

Fixed point · a decimal point bolted in place

Split 32 bits: 16 for the whole part, 16 for the fraction (“16.16”):

- Smallest step: $2^{-16} \approx 0.000015$ — everywhere on the line
- Largest value: $2^{15} \approx 32,768$
- Example: $6.25 = 0000000000000110.0100000000000000_2$

Simple, fast, and used in DSP chips and retro game consoles. **But look at that range**: nothing above 33k, nothing (nonzero) below 0.000015 — and both limits are hard walls.

Why ML kills fixed point · dynamic range

One training run of one neural network routinely contains, **simultaneously**:

Quantity	Typical magnitude
probability of a rare token	10^{-30} and below
a small gradient	10^{-8}
a learning rate	10^{-3}
activations, logits	1 to 10^{4}
a loss spike, a softmax numerator	10^{10} and beyond

That's 40+ orders of magnitude in one program. A fixed decimal point serves ~9 of them. We need a number format where the point floats.

The rescue idea is 400 years old · scientific notation

Chemists never write Avogadro's number as $602,214,076,000,000,000,000,000$. They write:

$$6.022 \times 10^{23}$$

- **A few significant digits** (the *mantissa*: 6.022) — the precision
- **A magnitude field** (the *exponent*: 23) — controls the range

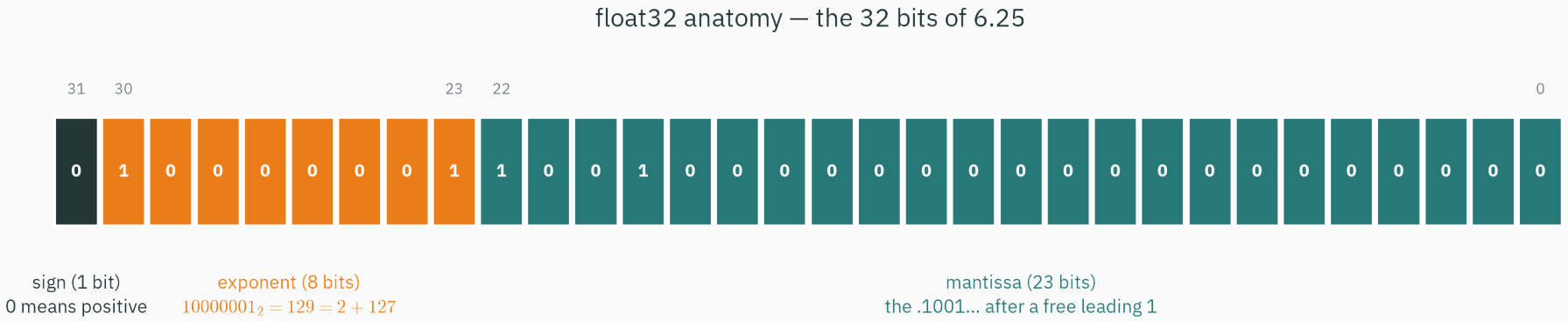
Same digit budget describes atoms (10^{-27} kg) or stars (10^{30} kg): the point *floats* to where it's needed.

A floating-point number is scientific notation, in base 2, squeezed into 32 bits. That is the whole idea — the rest is bookkeeping.

IEEE-754: the 32 bits, dissected

The anatomy of a float32

V VISUAL



$$\text{value} = (-1)^0 \times 1.1001_2 \times 2^{129-127} = 1.5625 \times 4 = 6.25$$

The decoding rule

$$\text{value} = (-1)^{\text{sign}} \times \underbrace{1.\text{mantissa}}_{24 \text{ significant bits}} \times 2^{\text{exponent} - 127}$$

- **Sign** (1 bit): 0 is positive, 1 is negative
- **Exponent** (8 bits): stored with a **bias of 127**, so the field $[1, 254]$ covers powers 2^{-126} to 2^{127} — no separate sign needed for the exponent
- **Mantissa** (23 bits): binary normalized numbers always start "1.", so the leading 1 is **not stored** — a free 24th bit of precision

---

Two exponent patterns are reserved: all-zeros (zero & *subnormals*, later ★★★) and all-ones (`inf` & `nan`, soon).

Worked encoding · 6.25, step by step

D DERIVATION

Step 1 — binary. $6.25 = 4 + 2 + 0.25 = 110.01_2$

Step 2 — normalize. $110.01_2 = 1.1001_2 \times 2^2$

Step 3 — sign. positive → $s = 0$

Step 4 — exponent. $2 + 127 = 129 = 10000001_2$

Step 5 — mantissa. drop the leading "1." → 10010000000000000000000

$$0 \quad 10000001 \quad 10010000000000000000000 = \texttt{0x40C80000}$$

Tutorial 1's worksheet has you do this by hand for -0.75, 13.5, and one nasty one...

Worked encoding · 0.1 — and the wheels come off

D DERIVATION

Multiply by 2, harvest the integer bit, repeat:

$$0.1 \xrightarrow{\times 2} \underline{0}.2 \xrightarrow{\times 2} \underline{0}.4 \xrightarrow{\times 2} \underline{0}.8 \xrightarrow{\times 2} \underline{1}.6 \xrightarrow{\times 2} \underline{1}.2 \xrightarrow{\times 2} \underline{0}.4\cdots$$

$$0.1 = 0.0\overline{0011}_2 = 0.0001100110011001\ldots_2 \quad \text{— repeats forever!}$$

Like $\frac{1}{3} = 0.333\ldots$ in decimal: a perfectly clean fraction, an **infinite** expansion in the wrong base. The mantissa holds 23 bits, so the machine must **round**:

```
>>> from decimal import Decimal
>>> Decimal(float(np.float32(0.1)))
Decimal('0.100000001490116119384765625')
```

There is no 0.1 in floating point. There never was.

Why `0.1 + 0.2 != 0.3`

What float64 *actually stores* (every digit below is exact):

You typed	The machine stored
`0.1`	0.1000000000000000055511151231257827…
`0.2`	0.2000000000000000111022302462515654…
`0.1 + 0.2`	0.3000000000000000444089209850062616…
`0.3`	0.2999999999999999888977697537484345…

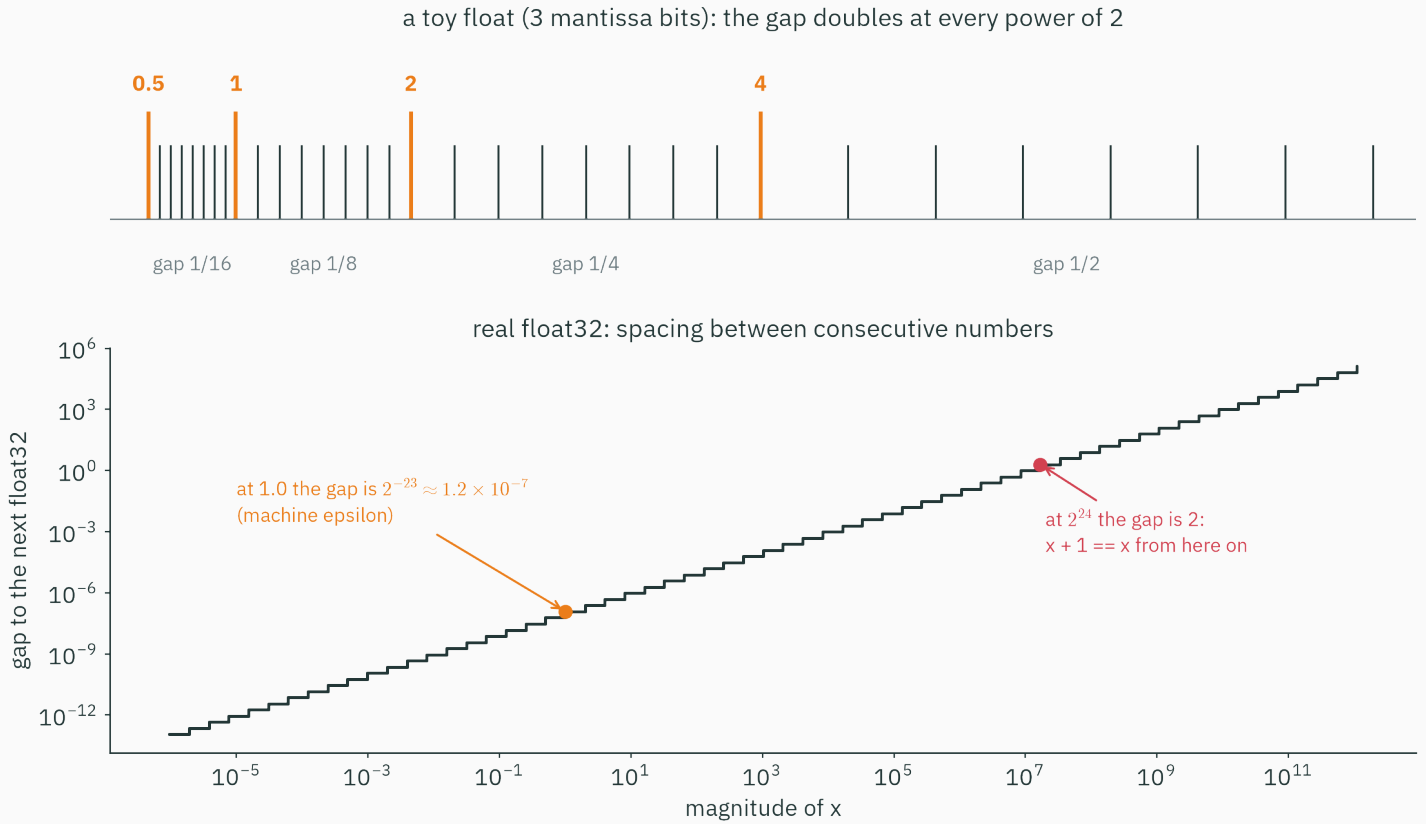
The two calculations produce different adjacent floating-point values, so `==` returns `False`.

Never compare floats with `==`. Use `np.isclose(a, b)` / `math.isclose(a, b)` — every test suite you write in this course will.

The float number line

The number line, as the machine sees it

V VISUAL



Machine epsilon · the gap next to 1.0

Machine epsilon ε = the gap between 1.0 and the next representable float.

$$\varepsilon_{\text{float32}} = 2^{-23} \approx 1.19 \times 10^{-7} \qquad \varepsilon_{\text{float64}} = 2^{-52} \approx 2.22 \times 10^{-16}$$

Anything smaller than about $\varepsilon/2$, added to 1, simply **vanishes**:

```
>>> np.float32(1.0) + np.float32(1e-8) == np.float32(1.0)
True                        # 1e-8 fell into the gap

>>> 1.0 + 1e-8 == 1.0       # float64's gap is much smaller
False
```

Rule of thumb: float32 carries **~7 reliable decimal digits**; float64 carries ~16.

At 2^{24}, adding 1 does nothing

The gap doubles at every power of 2. By $2^{24} = 16,777,216$ the gap between neighboring float32s is **2** — so $+1$ rounds right back:

```
>>> x = np.float32(16_777_216)      # 2**24
>>> x + np.float32(1) == x
True
>>> np.spacing(x)                   # the gap at x
2.0
```

A float32 counter that increments by 1 **stops counting at 16.7 million** — silently. Count in integers; accumulate long sums (losses, metrics) in float64.

Rounding · one tiny lie per operation

Each arithmetic result is computed exactly, then **rounded** to the nearest float:

$$\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta), \quad |\delta| \leq \frac{\varepsilon}{2}$$

- Default mode: **round to nearest, ties to even** — ties go to the even last bit, so up/down rounds don't accumulate a bias
- Other IEEE modes exist (toward 0, toward $\pm\infty$) — used for interval arithmetic, not our concern

One operation = one tiny, *bounded* lie (10^{-8} relative in float32). The drama is never one lie — it's how lies compound.

When arithmetic escapes the range · `inf` and `nan`

IEEE-754 doesn't crash on impossible arithmetic — it returns special values:

Expression (NumPy)	Result	Meaning
`1.0 / 0.0`	`inf`	overflow / true limit
`-1.0 / 0.0`	`-inf`	
`0.0 / 0.0`	`nan`	"not a number" — no defensible value
`inf - inf`, `inf / inf`	`nan`	
`np.log(-1.0)`, `np.sqrt(-1.0)`	`nan`	

NaN is contagious: any operation touching `nan` returns `nan`. And `nan == nan` is `False` — the only value not equal to itself (`x != x` is the classic NaN test; prefer `np.isnan`). One NaN at step 618 → *every* weight is NaN by step 619.

Overflow arrives shockingly early

The range boundary isn't some astronomical corner case. For `exp`:

Computation	float32 (max ≈ 3.4×10^{38})	float64 (max ≈ 1.8×10^{308})
`exp(88)`	1.65×10^{38} ✓	fine
`exp(89)`	**inf**	fine
`exp(709)`	inf	8.2×10^{307} ✓
`exp(710)`	inf	**inf**
`exp(-104)`	**0.0** (underflow)	fine

In float32, e^z overflows for z near 89; the opening logits around 1000 therefore overflow.

Catastrophic cancellation

Subtraction of near-equals is a digit shredder

Work in 8 significant decimal digits (like a float32-ish machine):

$$1.2345678 - 1.2345677 = 0.0000001$$

Both inputs had **8** correct digits. The result has **1** — and if the inputs were themselves rounded (they always are), that surviving digit is *pure noise*.

Adding same-sign numbers keeps relative error tame. Subtracting nearly equal numbers cancels the trustworthy leading digits and promotes the rounded-off garbage to the front.

Nothing "overflows" — the answer is just quietly wrong.

Worked example 1: the quadratic formula

D DERIVATION

Solve $x^2 - 10000x + 1 = 0$ in float32. The small root, two algebraically identical ways:

Naive (subtract near-equals: $b \approx \sqrt{b^2 - 4}$):

$$x = \frac{b - \sqrt{b^2 - 4}}{2} = \frac{10000 - 9999.9998\ldots}{2} \underset{\text{float32}}{\Rightarrow} \mathbf{0.0}$$

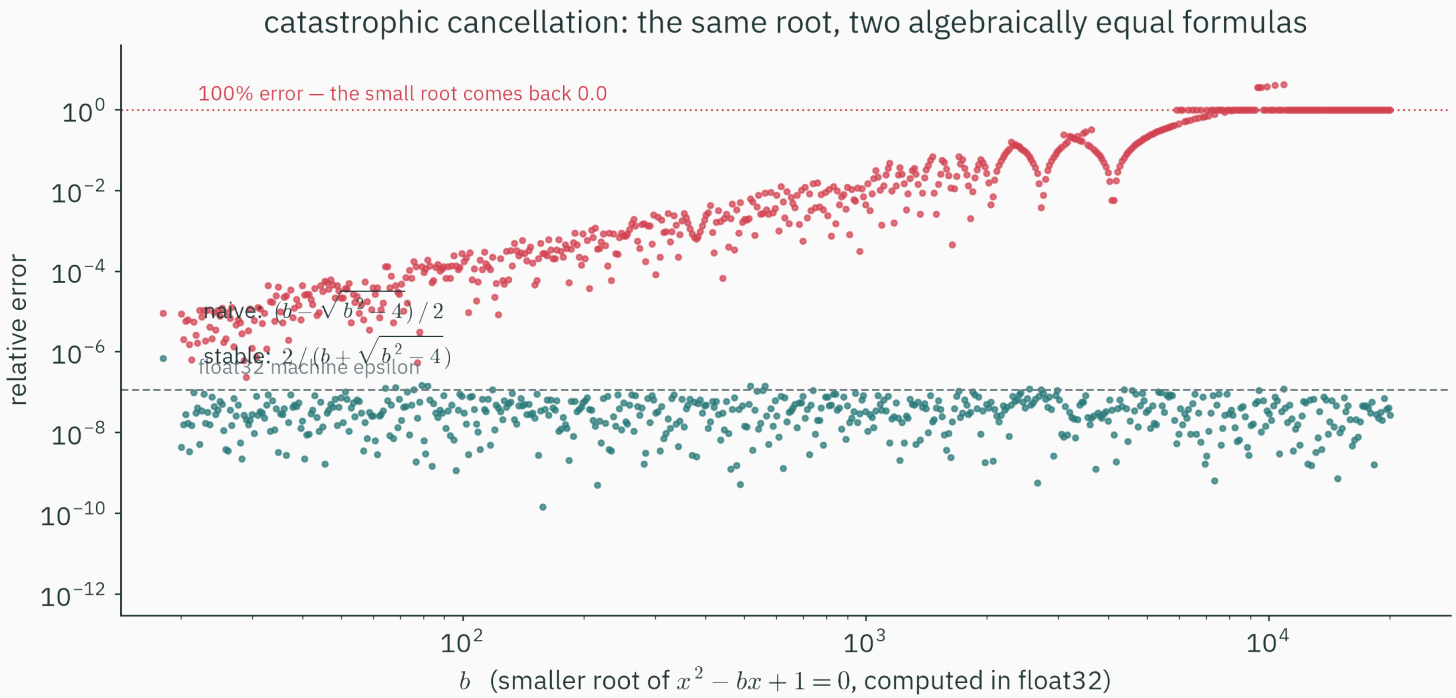
Stable (rationalize — same numbers, now *added*):

$$x = \frac{2}{b + \sqrt{b^2 - 4}} = \frac{2}{19999.9998} \underset{\text{float32}}{\Rightarrow} \mathbf{1.0000 \times 10^{-4}}\ \checkmark$$

The naive formula returned **zero** — a 100% error — with no warning of any kind.

The two formulas, head to head

V VISUAL



Same math on paper. In float32, one drifts to 100% error; the other sits at machine epsilon.

Worked example 2: two variance formulas

Textbooks give two equal formulas for variance. Try both on `[10000.0, 10000.1, 10000.2]` (true variance ≈ **0.00667**), in float32:

One-pass $\mathbb{E}[X^2] - (\mathbb{E}[X])^2$: two huge, nearly equal numbers —

$$100001992.0 - 100002011.7 = -19.7 \quad \text{(a negative variance!)}$$

Two-pass $\mathbb{E}\left[(X - \bar{X})^2\right]$: subtract *first*, while numbers are small —

$$\text{mean of } \{(-0.1)^2, 0^2, (0.1)^2\} = 0.00668 \checkmark$$

Near 10^8, float32's grid spacing is 8 — the true answer is a thousand times *smaller than the gap between representable numbers* there. Keep intermediate quantities small.

The ML fixes

Fix 1 · probabilities live in log-space

A language model scores a 1000-character text; each character has probability ≈ 0.03:

$$P(\text{text}) = \prod_{i=1}^{1000} p_i \approx 0.03^{1000} \approx 10^{-1523} \underset{\text{even float64}}{\Rightarrow} \mathbf{0.0}$$

In log-space — products become sums, tiny becomes ordinary:

$$\log P(\text{text}) = \sum_{i=1}^{1000} \log p_i \approx 1000 \times (-3.51) = -3507 \quad \checkmark \text{ a boring, safe float}$$

Multiply probabilities → add log-probabilities. Every loss you will ever meet is a *log*-likelihood — floating point demands it (L14 shows the deeper reason).

Fix 2 · log-sum-exp, derived in three lines

D DERIVATION

In log-space we still must *normalize*: compute $\log \sum_i e^{x_i}$ — but the e^{x_i} overflow! Pull out the max $m = \max_i x_i$:

$$\log \sum_i e^{x_i} = \log \sum_i e^m e^{x_i - m} = \log \left(e^m \sum_i e^{x_i - m} \right) = m + \log \sum_i e^{x_i - m}$$

Every exponent $x_i - m \leq 0$, so every $e^{x_i - m} \in (0, 1]$: **nothing can overflow** — and the largest term is exactly 1, so no total underflow either.

Check on the opening scores $[1000, 1001, 1002]$:

$$\mathrm{LSE} = 1002 + \log(e^{-2} + e^{-1} + e^{0}) = 1002 + \log(1.503) = 1002.41 \quad \checkmark$$

Fix 3 · the stable softmax

Softmax is **shift-invariant** — multiply top and bottom by e^{-c}:

$$\mathrm{softmax}(\boldsymbol{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{z_i - c}}{\sum_j e^{z_j - c}} \quad \text{for any } c$$

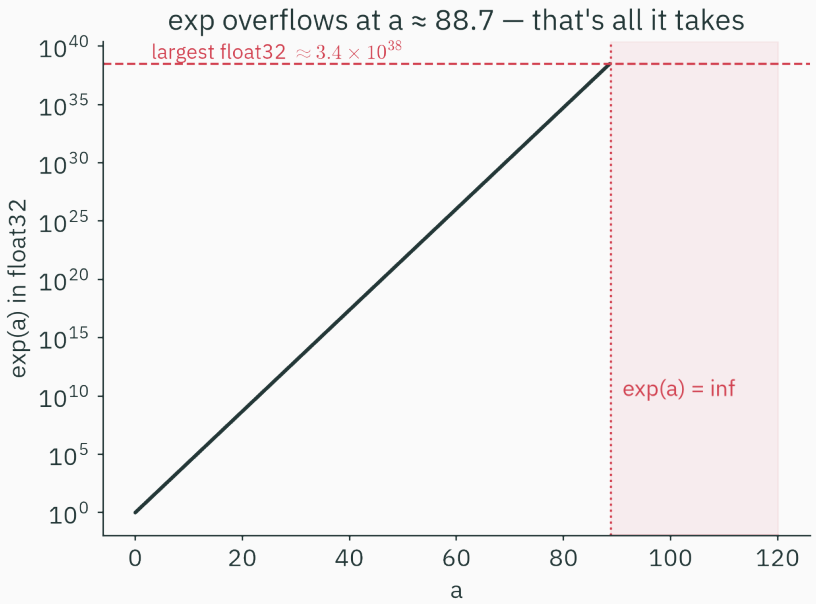
Choose $c = \max_j z_j$. For $\boldsymbol{z} = [1000, 1001, 1002]$:

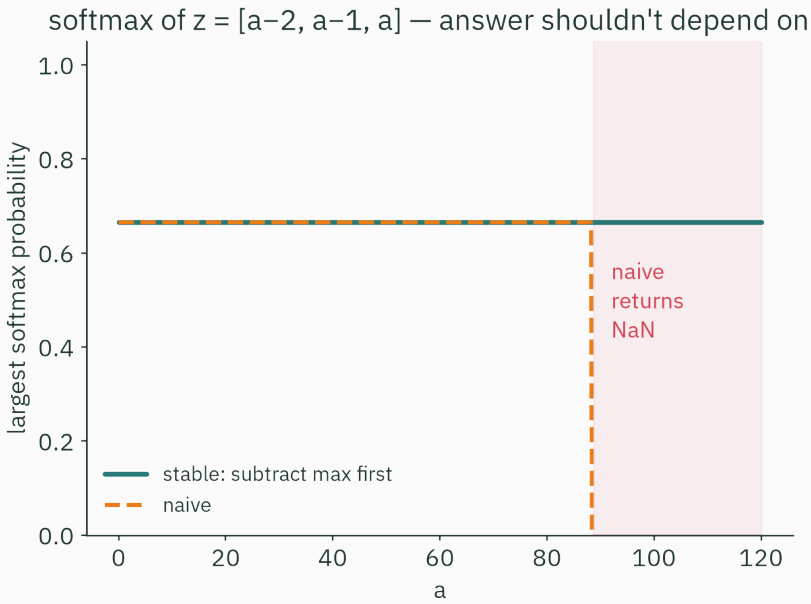
	z_i	$z_i - 1002$	$e^{z_i - 1002}$	**probability**
naive	$e^{1000} =$ `inf`		`inf / inf`	**nan**
stable		$-2, -1, 0$	$0.135, 0.368, 1.0$	$0.090, 0.245, \mathbf{0.665}$ ✓

Identical mathematics. One works on real hardware; the other burned down a training run.

The fix, in pictures

V VISUAL





The fixes, in code

```
def softmax_naive(z):
    e = np.exp(z)                # inf for z ~ 89+  → nan
    return e / e.sum()

def softmax_stable(z):
    e = np.exp(z - z.max())      # largest exponent is now 0
    return e / e.sum()           # same answer, can't overflow

def logsumexp(x):
    m = x.max()
    return m + np.log(np.exp(x - m).sum())
```

```
>>> softmax_stable(np.array([1000., 1001., 1002.]))
array([0.09003057, 0.24472847, 0.66524096])
```

Five lines you will reuse in Tutorial 1, in L14 (MLE), in L24 (cross-entropy), and in L26′s language model.

Stable softmax avoids the overflow

What actually happened at step 618:

1. The model got confident → logits grew past **±89** in float32
2. `exp(logit)` → `inf` → `inf / inf` → **nan** in the softmax
3. NaN loss → NaN gradients → NaN weights, **everywhere, permanently**

This is why PyTorch's `CrossEntropyLoss` takes **raw logits**, not probabilities: it fuses log-softmax into the loss and applies *exactly* the subtract-the-max trick you just derived. The fix for a billion-dollar training run is one line of first-year algebra.

Precision in deep learning

The four formats that run the AI world

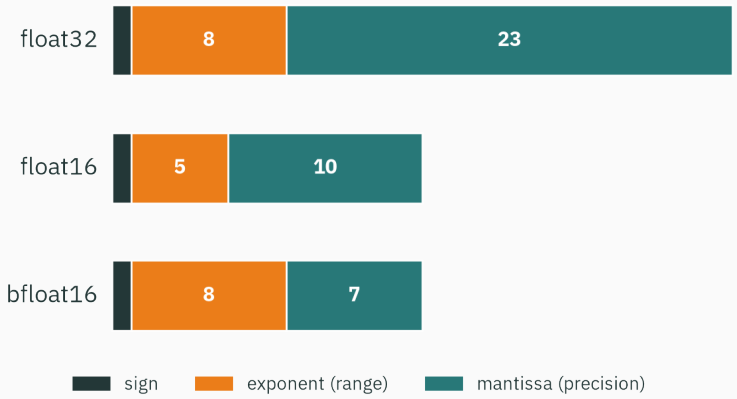
Format	Bits	Exp	Mant	Max value	Machine ε	~Digits
float64	64	11	52	1.8×10^{308}	2.2×10^{-16}	15.9
float32	32	8	23	3.4×10^{38}	1.2×10^{-7}	7.2
float16	16	5	10	**65 504**	9.8×10^{-4}	3.3
bfloat16	16	8	7	3.4×10^{38}	7.8×10^{-3}	2.4

Halving the bits doubles the training throughput and halves the memory — modern GPUs are *built* around 16-bit math. But look at float16's max value…

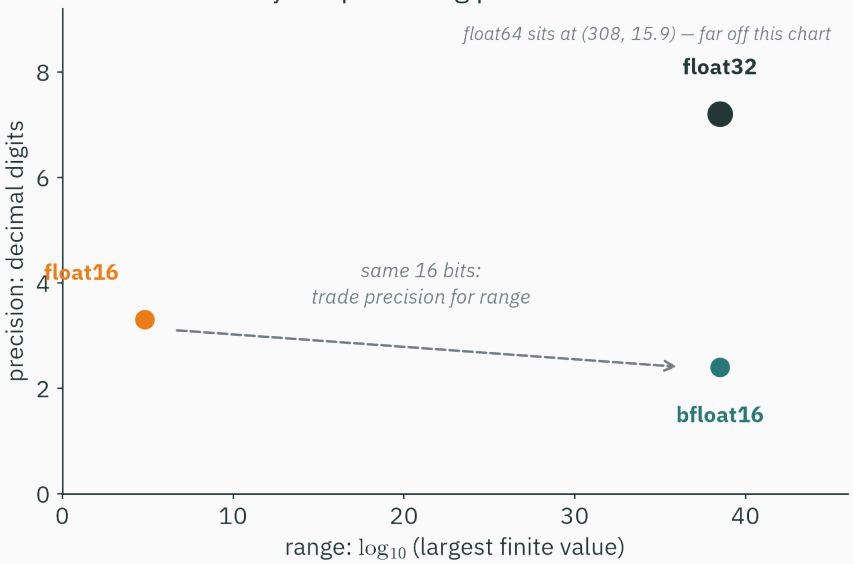
Range vs precision · the 16-bit dilemma

V VISUAL

bit budgets — bfloat16 = float32 with the mantissa chopped



why deep learning picked bfloat16



Why deep learning picked bfloat16

float16 spent its bits on precision and starved the exponent — `exp(12)` already overflows its 65,504 ceiling. **bfloat16 keeps float32's 8 exponent bits** and pays with mantissa:

	float16	**bfloat16**
`exp(12)` ≈ 162,755	**inf** — training dies	1.63×10^5 ✓
copy a float32's range?	no — overflow risk everywhere	yes — same exponent, just chop
digits of precision	3.3	2.4

DL chose range over precision: overflow makes NaNs (fatal), while low-precision noise mostly averages out across millions of SGD updates. Losing digits is survivable. Losing the exponent is not.

Checkpoint: you are now floating-point literate

Q QUESTION

Predict all three, then commit.

A. `np.exp(np.float16(12.0))` returns…?

B. `np.float32(16_777_216) + np.float32(1.0)` returns…?

C. From the start of class: is `0.5 + 0.25 == 0.75` `True` or `False`?

D. (bonus) why is `nan == nan` `False`?

Answers: floating-point literacy

A ANSWER

A. `inf` — $e^{12} \approx 162,755 > 65,504$, float16's ceiling. (bfloat16 shrugs: $\approx 1.6 \times 10^5$.)

B. `16777216.0` — unchanged. At 2^{24} the float32 gap is 2, so $+1$ rounds back down.

C. True! $0.5 = 2^{-1}$, $0.25 = 2^{-2}$, $0.75 = 2^{-1} + 2^{-2}$ — all *exactly* representable in binary. Floats aren't sloppy; they are **exact binary fractions**. Trouble only starts when your decimal (0.1) has no finite binary form.

D. NaN means "no defensible value" — two undefined results can't be declared equal. `x != x` is the classic NaN test.

Practice problems

Try on paper; verify in the Tutorial 1 notebook.

1. Encode -0.75 as float32 (sign, exponent field, first mantissa bits).
2. What is the largest odd integer float32 represents exactly? (Hint: where does the gap reach 2?)
3. Compute $\mathrm{logsumexp}([-1000, -1001])$ by hand. What would the naive computation return?
4. Explain why `(0.1 + 0.2) + 0.3 != 0.1 + (0.2 + 0.3)` in float64 — what algebraic law do floats break?
5. Predict the sign and rough size of the error when the one-pass variance formula meets `[1e6, 1e6 + 1, 1e6 + 2]` in float32.
6. Your float16 training run dies at `exp(12)`. A colleague proposes bfloat16. What exactly changes, and what do you give up?

⭐⭐⭐ Kahan summation

★ OPTIONAL

Add `np.float32(0.1)` ten million times. True answer: 10^6. A naive loop returns…

`1087937.0` — off by 8.8%

Once the sum reaches 10^6, the grid gap is 0.0625: each $+0.1$ rounds badly, ten million times, *in the same direction*.

```
s, c = 0.0, 0.0
for x in xs:
    y = x - c           # re-inject the error we still owe
    t = s + y           # big + small: low digits of y get dropped...
    c = (t - s) - y     # ...(t - s) - y recovers exactly what was dropped
    s = t
```

Kahan's compensated sum tracks the rounding error in `c` and repays it — error stays ~ε **independent of n**. (`np.sum` fights the same war with pairwise summation.)

⭐⭐⭐ Subnormals: the fine print near zero

★ OPTIONAL

The smallest *normal* float32 is $2^{-126} \approx 1.18 \times 10^{-38}$. Without further tricks there'd be a **moat around zero**, and `x - y == 0` could be true for `x != y`(!).

Subnormals: when the exponent field is all zeros, drop the implicit leading 1 and let the mantissa alone carry the value — filling the moat with evenly spaced numbers down to $2^{-149} \approx 1.4 \times 10^{-45}$ ("gradual underflow").

The catch: some hardware handles subnormals in microcode — **orders of magnitude slower**. GPUs and DL runtimes often enable *flush-to-zero* modes that trade this correctness for speed. If a kernel inexplicably slows down as values shrink toward zero: suspect subnormals.

Lecture 2 — summary

- **Fixed formats fail ML**: one program spans 40+ orders of magnitude → the point must float.
- **IEEE-754 float32** = sign + biased exponent (127) + mantissa with a free leading 1; 6.25 encodes exactly, 0.1 *cannot* — hence `0.1 + 0.2 != 0.3`.
- **The float line is unevenly spaced**: gaps double at each power of 2; $\varepsilon_{32} \approx 10^{-7}$; at 2^{24}, $x + 1 == x$.
- **Special values**: overflow → `inf`, indeterminate → `nan`; NaN propagates and `nan != nan`.
- **Catastrophic cancellation**: subtracting near-equals shreds digits — rewrite the formula.
- **The ML fixes**: log-space probabilities, log-sum-exp, subtract-the-max softmax.
- **bfloat16** = float32's range at half the bits: DL trades precision for range.

Lecture 2 — summary

Read before L3 — Goldberg (1991), §1–2 · Solomon, *Numerical Algorithms*, Ch. 1–2 (free PDF). **Tutorial 1** this week: IEEE-754 by hand, then break `0.1 + 0.2`, cancellation, and softmax yourself in NumPy — and fix them.

Computers don’t do real numbers — they do ~4 billion of them, unevenly spaced.

Next: **vectors** — how direction and displacement explain `king − man + woman ≈ queen`.